ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Membre de HONORIS UNITED UNIVERSITIES

MÉMOIRE DE PROJET

CONCEPTION ET DÉVELOPPEMENT D'UN MODULE MÉTIER SOUS L'ERP ODOO 17

Sujet : La Boîte à Idées Collaborative

Réalisé par :

Bouziane Achraf

Filière : IIR - 5ème Année

Encadré par :

M. AIT DAOUD

Année Universitaire 2025 - 2026

Résumé

Ce rapport présente la conception, le développement et le déploiement d'un module personnalisé nommé "Boîte à Idées" sur la plateforme ERP Odoo version 17. Ce projet s'inscrit dans le cadre de la formation en ingénierie logicielle et vise à répondre à une problématique de gestion de l'innovation participative au sein des entreprises.

L'objectif principal était de digitaliser le processus de collecte et de traitement des idées proposées par les collaborateurs. Le module permet de centraliser les propositions, de les catégoriser et de suivre leur validation à travers un workflow structuré (Brouillon, Soumise, Validée, Refusée).

Sur le plan technique, le projet a été réalisé en adoptant une architecture conteneurisée basée sur Docker et Docker Compose, garantissant l'isolation des services (Base de données PostgreSQL 16 et Serveur Odoo 17) et la reproductibilité de l'environnement de développement . Le développement backend a été effectué en Python en respectant l'architecture MVC propre à Odoo, tandis que l'interface utilisateur a été conçue via les vues XML .

Les résultats obtenus démontrent la viabilité de la solution : le module est fonctionnel, sécurisé via des listes de contrôle d'accès (ACL), et parfaitement intégré à l'écosystème Odoo existant. Ce travail a permis de valider des compétences clés en développement ERP et en administration système.

Mots-clés : ERP, Odoo 17, Docker, Python, PostgreSQL, Workflow, Innovation Participative.

Abstract

This report presents the design, development, and deployment of a custom module named "Idea Box" on the Odoo 17 ERP platform. This project is part of the software engineering curriculum and aims to address the challenge of managing participatory innovation within companies.

The main objective was to digitize the process of collecting and processing ideas proposed by employees. The module allows for the centralization of proposals, their categorization, and the tracking of their validation through a structured workflow (Draft, Submitted, Validated, Refused).

Technically, the project was carried out using a containerized architecture based on Docker and Docker Compose, ensuring service isolation (PostgreSQL 16 Database and Odoo 17 Server) and reproducibility of the development environment. Backend development was done in Python following Odoo's MVC architecture, while the user interface was designed using XML views.

The results demonstrate the viability of the solution : the module is functional, secured via Access Control Lists (ACL), and seamlessly integrated into the existing Odoo ecosystem. This work validated key skills in ERP development and system administration.

Keywords : ERP, Odoo 17, Docker, Python, PostgreSQL, Workflow, Participatory Innovation.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude et mes sincères remerciements à mon encadrant, **Monsieur AIT DAOUD**, pour sa disponibilité, ses conseils avisés et la qualité de son encadrement tout au long de ce projet. Ses directives précieuses concernant l'architecture technique d'Odoo et la conteneurisation Docker m'ont permis de mener à bien ce travail dans les meilleures conditions.

Je souhaite également remercier l'ensemble du corps professoral de **l'École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la qualité de l'enseignement dispensé et pour leur engagement à nous fournir les compétences nécessaires pour relever les défis technologiques actuels.

Mes remerciements s'adressent aussi à tous ceux qui ont contribué, de près ou de loin, à l'élaboration de ce projet, notamment mes camarades de promotion pour nos échanges fructueux et leur soutien constant.

Enfin, je remercie ma famille pour ses encouragements indéfectibles et sa patience tout au long de mon parcours académique.

Table des matières

Résumé **1**

Remerciements **3**

1 Introduction Générale **7**
1.1 Contexte du projet 7
1.2 Objectifs du projet 7

2 Analyse et Conception du Système **8**
2.1 Introduction et Méthodologie 8
2.2 Analyse des Besoins et Spécifications Fonctionnelles 8
2.2.1 Critique de l'existant et Justification 8
2.2.2 Identification des Acteurs 9
2.2.3 Dictionnaire des Fonctionnalités (User Stories) 10
2.3 Conception des Données (Data Modeling) 10
2.3.1 Entité Principale : BoiteIdee 10
2.3.2 Structure de la Base de Données (PostgreSQL) 11
2.4 Conception des Traitements et Workflow 12
2.4.1 États du Système 12
2.4.2 Matrice des Transitions 12
2.5 Analyse de la Sécurité et Droits d'Accès 13
2.5.1 Fichier ir.model.access.csv 13
2.6 Contraintes Non-Fonctionnelles 13
2.6.1 Ergonomie (UI/UX) 14
2.6.2 Performance 14
2.6.3 Maintenabilité 14
2.7 Conclusion de l'Analyse 14

3 Architecture Technique et Docker **15**
3.1 L'approche Docker 15

3.2 Le fichier docker-compose.yml 15
3.3 Gestion des Volumes 15

4 Réalisation du Module : Étape par Étape 16
4.1 Étape 1 : Préparation de l'arborescence 16
4.2 Étape 2 : Le Manifeste (__manifest__.py) 16
4.3 Étape 3 : Le Modèle de Données (Python) 17
4.4 Étape 4 : La Sécurité (CSV) 18
4.5 Étape 5 : L'Interface Utilisateur (XML) 18
4.5.1 Vue Liste (Tree View) 18
4.5.2 Vue Formulaire (Form View) 19

5 Déploiement, Tests et Résolution d'Incidents 20
5.1 Procédure de Déploiement 20
5.2 Tests Fonctionnels 21
5.2.1 Test 1 : Création d'une idée 21
5.2.2 Test 2 : Validation (Workflow) 21

6 Conclusion Générale et Bilan 22
6.1 Bilan Technique et Architectural 22
6.2 Bilan Fonctionnel et Ergonomique 23
6.3 Apports Personnels et Compétences Acquises 23

7 Perspectives d'Évolution et Industrialisation 24
7.1 Évolutions Fonctionnelles Avancées 24
7.1.1 Intégration Transverse (Cross-Module) 24
7.1.2 Communication et Collaboration (Chatter) 24
7.1.3 Gamification et Engagement 25
7.2 Améliorations Techniques et Sécurité 25
7.2.1 Sécurité Granulaire (Record Rules) 25
7.2.2 Portail Web (Frontend) 25
7.3 Stratégie de Déploiement (DevOps) 25
7.3.1 Intégration Continue (CI/CD) 25
7.3.2 Hébergement Cloud 26

Bibliographie 27

Table des figures

4.1 Arborescence des fichiers du projet 17

5.1 Module Boîte à Idées prêt à être installé 20
5.2 Saisie d'une nouvelle idée 21
5.3 Liste des idées avec différents statuts 21

Chapitre 1

Introduction Générale

1.1 Contexte du projet

La transformation numérique des entreprises ne se limite pas à la digitalisation des processus administratifs. Elle englobe également la capacité d'une organisation à mobiliser l'intelligence collective de ses collaborateurs. Dans ce contexte, la gestion des idées (idéation) devient un processus critique. Les méthodes traditionnelles (boîtes physiques, emails dispersés, fichiers Excel partagés) montrent rapidement leurs limites : perte d'informations, absence de suivi, manque de transparence et démotivation des employés. L'utilisation d'un ERP (Enterprise Resource Planning) comme Odoo permet de centraliser ce processus au sein du même outil que celui utilisé pour la production, la vente ou les ressources humaines.

1.2 Objectifs du projet

Ce projet a pour but de développer un module personnalisé "Boîte à Idées" sur Odoo 17. Les objectifs sont multiples :

- **Technique :** Appréhender l'architecture MVC d'Odoo, le langage Python, et le système de vues XML.
- **Infrastructure :** Maîtriser le déploiement conteneurisé via Docker et Docker Compose.
- **Fonctionnel :** Fournir un workflow complet de validation (Brouillon → Validé).

Chapitre 2

Analyse et Conception du Système

2.1 Introduction et Méthodologie

La phase d'analyse et de conception est une étape charnière dans le cycle de vie de développement logiciel. Elle permet de traduire les besoins métiers, souvent exprimés de manière informelle, en spécifications techniques précises et implémentables au sein de l'ERP Odoo. Dans le cadre de ce projet, nous avons adopté une approche analytique basée sur la décomposition fonctionnelle. L'objectif n'est pas seulement de créer un formulaire de saisie, mais de concevoir un système d'information cohérent qui respecte les standards de développement Odoo (architecture MVC) et les contraintes de déploiement liées à Docker.

Nous avons structuré cette phase en trois temps :

1. **L'analyse de l'existant et des besoins :** Comprendre pourquoi le système actuel est défaillant et ce que le nouveau module doit apporter.
2. **La modélisation des données :** Définir les entités, leurs attributs et les contraintes d'intégrité, en tenant compte des types de champs spécifiques à l'ORM Odoo (Char, Text, Selection, Date).
3. **La modélisation des traitements :** Définir le cycle de vie de l'information (Workflow) et les règles de gestion associées à chaque transition d'état.

2.2 Analyse des Besoins et Spécifications Fonctionnelles

2.2.1 Critique de l'existant et Justification

Dans de nombreuses structures, la gestion des idées repose sur des processus informels. Les collaborateurs envoient des emails à leur hiérarchie ou déposent des suggestions papier. Cette approche présente des limites majeures qui justifient le développement de notre module :

— **Perte d'information :** Sans base de données centralisée, les idées se perdent dans les boîtes emails ou sont oubliées.
— **Opacité du traitement :** L'employé ne sait jamais si son idée a été lue, étudiée ou rejetée.
— **Absence de statistiques :** Il est impossible de savoir combien d'idées ont été soumises par département ou par période.
— **Hétérogénéité des données :** Chaque employé formalise son idée différemment, rendant la comparaison difficile.

Notre module "Boîte à Idées" vient combler ces lacunes en imposant une structure de données unique et un processus de validation transparent.

2.2.2 Identification des Acteurs

Nous avons identifié trois typologies d'acteurs qui interagiront avec le système. La distinction de ces rôles est primordiale pour la future configuration de la sécurité (ACL).

1. Le Collaborateur (Utilisateur Standard)

C'est l'utilisateur final du module. Son rôle est d'alimenter le système.
— **Besoins :** Il doit pouvoir saisir une idée rapidement. Il a besoin de suivre l'avancement de ses propositions (savoir si elles sont validées ou refusées).
— **Contraintes :** Il ne doit pas pouvoir modifier les idées de ses collègues ni valider ses propres idées.

2. Le Manager (Gestionnaire)

C'est le garant du traitement des idées.
— **Besoins :** Il doit avoir une vue d'ensemble (Vue Liste) de toutes les idées soumises. Il est responsable du changement de statut (Validation ou Refus).
— **Contraintes :** Il doit pouvoir modifier une idée pour la corriger ou la compléter avant validation.

3. L'Administrateur Système

C'est le profil technique (Superuser Odoo).
— **Besoins :** Il a tous les droits. Il est responsable de la maintenance technique du module (mise à jour via Docker) et de la configuration des listes de sélection (catégories).

2.2.3 Dictionnaire des Fonctionnalités (User Stories)

Le périmètre fonctionnel du module a été arrêté comme suit :

SF01 - Création d'une fiche idée : Le système doit permettre la saisie d'un titre, d'une description détaillée et d'une catégorie. La date de création doit être automatique (date du jour) pour éviter les erreurs de saisie .

SF02 - Identification de l'auteur : Pour cette version, l'auteur est saisi manuellement sous forme de texte. Une évolution future pourrait lier ce champ à la table 'res.users' d'Odoo.

SF03 - Catégorisation : Le système doit proposer une liste fermée de catégories (RH, Technique, Autre) pour normaliser les données et faciliter les futurs tris.

SF04 - Gestion du cycle de vie (Workflow) : Une idée ne doit pas être statique. Elle doit évoluer d'un état "Brouillon" vers un état final ("Validée" ou "Refusée"). Ce changement d'état doit être visible visuellement (code couleur ou barre d'état).

SF05 - Consultation et Recherche : Les utilisateurs doivent pouvoir retrouver une idée facilement grâce à une vue liste et une barre de recherche standard Odoo.

2.3 Conception des Données (Data Modeling)

L'étape de modélisation des données consiste à définir comment les informations seront stockées physiquement dans la base de données PostgreSQL du conteneur 'db'. Odoo utilise un ORM (Object-Relational Mapping), nous définissons donc une classe Python qui sera traduite en table SQL.

2.3.1 Entité Principale : BoiteIdee

Le module repose sur une entité unique nommée techniquement 'boite.idee'. Voici le dictionnaire de données détaillé décrivant chaque attribut de cette entité :

Champ : name (Titre)

Type technique : `fields.Char`

Contrainte : `required=True`

Description : Il s'agit du titre court de l'idée. C'est le champ qui sera affiché par défaut dans les relations et les fils d'ariane. Il est obligatoire car une idée sans titre est inexploitable.

Champ : auteur (Proposant)

Type technique : `fields.Char`

Valeur par défaut : "Anonyme"

Description : Stocke le nom de la personne ayant émis l'idée. Le choix du type

Char (texte simple) plutôt que Many2one (lien vers la table Users) a été fait pour simplifier cette première version du module et permettre des saisies rapides sans créer de comptes utilisateurs complexes.

Champ : description (Détail)

Type technique : `fields.Text`

Description : Ce champ permet de stocker un texte long, potentiellement sur plusieurs lignes. Il sert à expliquer le "Pourquoi" et le "Comment" de l'idée. Contrairement au titre, il n'est pas limité en nombre de caractères.

Champ : date_creation (Date de soumission)

Type technique : `fields.Date`

Valeur par défaut : `fields.Date.context_today`

Description : Ce champ stocke la date d'émission. L'utilisation d'une fonction par défaut est cruciale pour l'expérience utilisateur : l'employé n'a pas à sélectionner la date manuellement, le système le fait pour lui.

Champ : categorie (Typologie)

Type technique : `fields.Selection`

Valeurs possibles :

— 'rh' : Ressources Humaines

— 'tech' : Amélioration Technique

— 'autre' : Divers

Description : Ce champ est stocké en base sous forme de clé (ex : 'rh') mais affiché à l'utilisateur sous forme de libellé (ex : 'Ressources Humaines'). Il permet de structurer la base de connaissances.

Champ : statut (État du workflow)

Type technique : `fields.Selection`

Valeur par défaut : 'brouillon'

Description : C'est le champ le plus important pour la logique métier. Il pilote la barre d'état (StatusBar) et détermine la couleur de la ligne dans la vue liste (décorateurs).

2.3.2 Structure de la Base de Données (PostgreSQL)

Grâce à l'ORM d'Odoo, cette classe Python générera automatiquement une table nommée `boite_idee` dans le conteneur PostgreSQL. Odoo ajoutera automatiquement les champs systèmes suivants (non définis dans notre code mais présents en base) :

— `id` (Integer) : Clé primaire unique.

— `create_uid` : ID de l'utilisateur créateur.

— `create_date` : Timestamp de création technique.
— `write_uid` : ID du dernier utilisateur ayant modifié.
— `write_date` : Timestamp de la dernière modification.

2.4 Conception des Traitements et Workflow

Au-delà du stockage des données, le module doit implémenter une logique métier dynamique. Nous avons modélisé cette logique sous forme d'une machine à états finis.

2.4.1 États du Système

Le processus de validation traverse quatre états distincts :

1. **État BROUILLON (Draft) :**
   — *Signification :* L'idée est en cours de rédaction. Elle n'est pas encore officielle.
   — *Actions possibles :* Modifier tous les champs, Supprimer la fiche, Passer à l'état "Soumise".
   — *Visibilité :* Visible par l'auteur et le manager.
2. **État SOUMISE (Submitted) :**
   — *Signification :* L'auteur a terminé sa rédaction et envoie l'idée pour arbitrage.
   — *Indicateur visuel :* Couleur bleue (info) dans la vue liste.
   — *Actions possibles :* Valider (Manager), Refuser (Manager), Remettre en brouillon.
3. **État VALIDÉE (Done) :**
   — *Signification :* L'idée a été acceptée et sera mise en œuvre. C'est un état final.
   — *Indicateur visuel :* Couleur verte (success) dans la vue liste.
   — *Actions possibles :* Aucune (Archivage).
4. **État REFUSÉE (Cancel) :**
   — *Signification :* L'idée n'est pas retenue. C'est un état final.
   — *Indicateur visuel :* Couleur rouge (danger) ou grisée.

2.4.2 Matrice des Transitions

Le tableau ci-dessous décrit les transitions autorisées entre les états. Cette logique sera implémentée via l'attribut `clickable=1` sur le widget StatusBar dans la vue XML, permettant aux utilisateurs de cliquer pour changer l'état.

État Source	**État Cible**	**Acteur Autorisé**
Brouillon	Soumise	Collaborateur
Soumise	Validée	Manager
Soumise	Refusée	Manager
Soumise	Brouillon	Manager (Correction)
Validée	-	(État Final)
Refusée	Brouillon	Manager (Réouverture)

2.5 Analyse de la Sécurité et Droits d'Accès

La sécurité est un aspect fondamental, même pour un module simple. Odoo gère la sécurité via deux mécanismes principaux : les groupes d'utilisateurs et les règles d'accès (ACL - Access Control Lists).

2.5.1 Fichier ir.model.access.csv

Pour ce module, nous devons définir explicitement qui a le droit de lire, écrire, créer ou supprimer des enregistrements dans la table `boite.idee`. Sans cette configuration, le module serait invisible pour les utilisateurs .

Nous avons défini une règle d'accès globale dans le fichier `security/ir.model.access.csv` :

— **Modèle concerné :** `model_boite_idee`
— **Groupe :** Tous les utilisateurs (champ vide pour `group_id`).
— **Permission Lecture (perm_read) :** 1 (Autorisé)
— **Permission Écriture (perm_write) :** 1 (Autorisé)
— **Permission Création (perm_create) :** 1 (Autorisé)
— **Permission Suppression (perm_unlink) :** 1 (Autorisé)

Note d'analyse : Dans un environnement de production réel, nous aurions créé deux groupes de sécurité distincts (ex : `group_idea_user` et `group_idea_manager`) pour restreindre la suppression et la validation aux seuls managers. Pour ce projet académique, nous avons opté pour une ouverture des droits afin de faciliter les tests fonctionnels.

2.6 Contraintes Non-Fonctionnelles

Outre les fonctionnalités métier, le système doit respecter des contraintes techniques imposées par l'architecture :

2.6.1 Ergonomie (UI/UX)

L'interface doit s'intégrer nativement dans le "Web Client" d'Odoo. Nous utiliserons les vues standards (Tree, Form) sans développement JavaScript spécifique (Owl framework), pour garantir une cohérence visuelle avec les autres modules (Ventes, Achats, etc.). L'utilisation de "Badges" colorés pour les statuts est requise pour permettre une lecture rapide de l'information.

2.6.2 Performance

Le module doit être léger. L'utilisation de champs relationnels complexes a été limitée. Le déploiement via Docker sur une base PostgreSQL dédiée garantit des temps de réponse optimaux.

2.6.3 Maintenabilité

Le code doit être modulaire. La séparation stricte entre le modèle (Python), la vue (XML) et la sécurité (CSV) permet une maintenance aisée. Si nous devons ajouter un champ "Coût estimé" demain, nous savons qu'il faudra modifier `idee.py` puis ajouter la balise correspondante dans `idee_views.xml`, sans risque de casser la logique existante.

2.7 Conclusion de l'Analyse

Cette phase d'analyse détaillée nous a permis de verrouiller le périmètre du projet. Nous avons défini une structure de données robuste (`boite.idee`), un workflow clair à 4 états, et une politique de sécurité initiale. Ces spécifications servent de fondation solide pour la phase de réalisation technique qui va suivre. ==========

Chapitre 3

Architecture Technique et Docker

3.1 L'approche Docker

Pour ce projet, nous n'avons pas installé Odoo directement sur la machine hôte. Nous avons utilisé Docker pour garantir l'isolation et la reproductibilité de l'environnement.

3.2 Le fichier docker-compose.yml

L'orchestration repose sur deux services. Voici la configuration utilisée :

3.3 Gestion des Volumes

Le point critique de cette architecture est la ligne `- ./addons:/mnt/extra-addons`. Cette instruction crée un "Bind Mount". Elle signifie que le dossier `addons` créé sur notre bureau Windows est directement accessible à l'intérieur du conteneur Linux Odoo dans le répertoire `/mnt/extra-addons`. C'est ce qui nous permet de développer le code sur notre machine et de le voir exécuté par le serveur.

Chapitre 4

Réalisation du Module : Étape par Étape

Ce chapitre détaille la procédure technique complète pour recréer le module.

4.1 Étape 1 : Préparation de l'arborescence

Nous avons commencé par créer la structure standard d'un module Odoo via le terminal.

```
# Dans le dossier du projet
mkdir -p addons/boite_idees
mkdir -p addons/boite_idees/models
mkdir -p addons/boite_idees/views
mkdir -p addons/boite_idees/security
touch addons/boite_idees/__init__.py
touch addons/boite_idees/__manifest__.py
```

4.2 Étape 2 : Le Manifeste (__manifest__.py)

Le manifeste est la carte d'identité du module. Il déclare les dépendances et les fichiers à charger .

```
{
    'name': 'Bo te    Id es',
    'version': '1.0',
    'summary': 'Module de gestion des id es collaboratives',
    'category': 'Productivity',
```

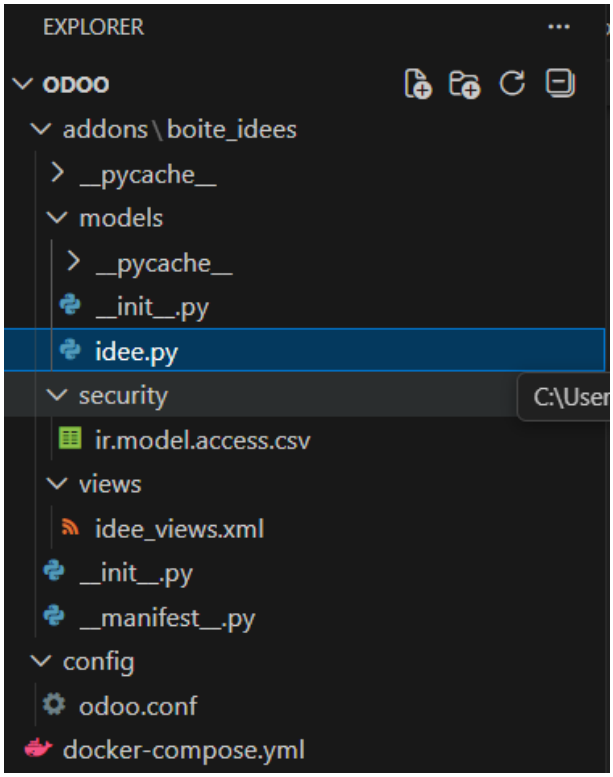


FIGURE 4.1 – Arborescence des fichiers du projet

```
    'author': 'Votre Nom',
    'depends': ['base'],
    'data': [
        'security/ir.model.access.csv',
        'views/idee_views.xml',
    ],
    'installable': True,
    'application': True,
}
```

Listing 4.1 – Code du Manifeste

4.3 Étape 3 : Le Modèle de Données (Python)

Nous avons créé le fichier `models/idee.py`. C'est ici que la logique métier réside .

```
from odoo import models, fields

class BoiteIdee(models.Model):
    _name = 'boite.idee'
```

```
    _description = 'Gestion des Id es'

    name = fields.Char(string='Titre', required=True)
    auteur = fields.Char(string='Auteur', default='Anonyme')
    description = fields.Text(string='Description')
    date_creation = fields.Date(string='Date', default=fields.Date.
   context_today)

    categorie = fields.Selection([
        ('rh', 'Ressources Humaines'),
        ('tech', 'Technique'),
        ('autre', 'Autre')
    ], string='Cat gorie', default='autre')

    statut = fields.Selection([
        ('brouillon', 'Brouillon'),
        ('validee', 'Valid e'),
        ('refusee', 'Refus e')
    ], string=' tat ', default='brouillon')
```

Listing 4.2 – Code du Modèle idee.py

Note : Ne pas oublier d'importer ce fichier dans `models/__init__.py`.

4.4 Étape 4 : La Sécurité (CSV)

Pour que le modèle soit accessible, nous avons défini les droits d'accès dans `security/ir.model.acce`

.

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,
   perm_unlink
access_boite_idee,boite.idee,model_boite_idee,,1,1,1,1
```

4.5 Étape 5 : L'Interface Utilisateur (XML)

Le fichier `views/idee_views.xml` contient la définition des menus et des vues .

4.5.1 Vue Liste (Tree View)

```xml
<record id="view_idee_tree" model="ir.ui.view">
    <field name="name">boite.idee.tree</field>
    <field name="model">boite.idee</field>
    <field name="arch" type="xml">
        <tree decoration-success="statut=='validee'">
            <field name="name"/>
            <field name="auteur"/>
            <field name="statut" widget="badge"/>
        </tree>
    </field>
</record>
```

4.5.2 Vue Formulaire (Form View)

```xml
<record id="view_idee_form" model="ir.ui.view">
    <field name="name">boite.idee.form</field>
    <field name="model">boite.idee</field>
    <field name="arch" type="xml">
        <form>
            <header>
                <field name="statut" widget="statusbar" options="{'
   clickable': '1'}"/>
            </header>
            <sheet>
                <group>
                    <field name="name"/>
                    <field name="categorie"/>
                </group>
            </sheet>
        </form>
    </field>
</record>
```

Chapitre 5

Déploiement, Tests et Résolution d'Incidents

5.1 Procédure de Déploiement

Une fois le code écrit, nous devons informer Odoo des changements.

1. **Redémarrage du conteneur :** Nécessaire pour charger le code Python.

```
    docker restart odoo_app

```

2. **Mise à jour des Apps :** Via l'interface web en mode développeur ("Update Apps List").
3. **Installation :** Clic sur "Activer" sur le module Boîte à Idées.

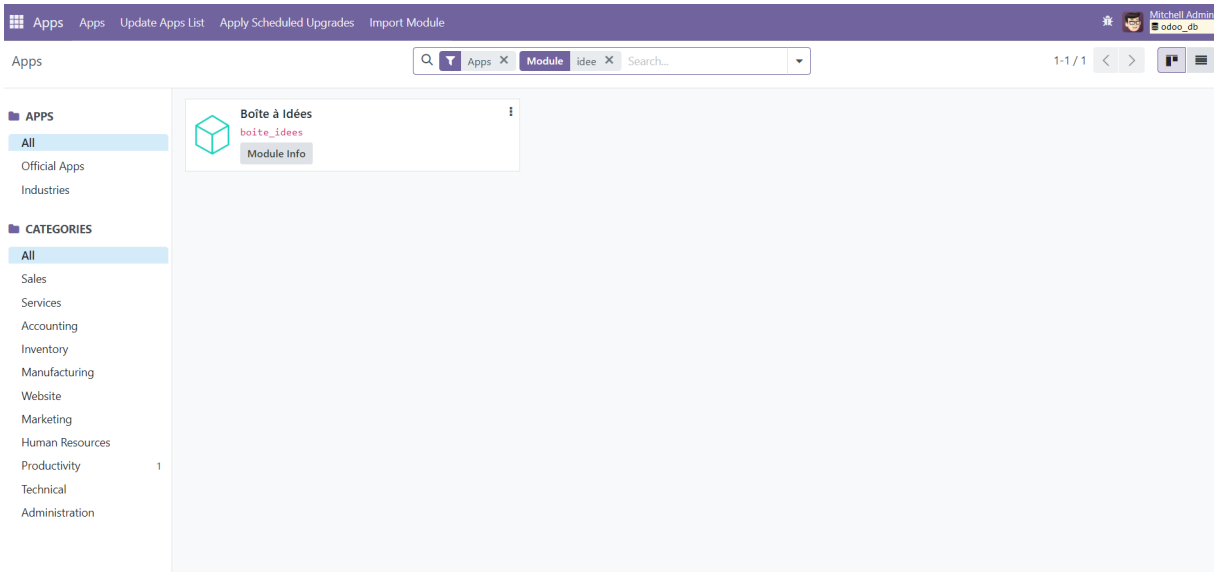


FIGURE 5.1 – Module Boîte à Idées prêt à être installé

5.2 Tests Fonctionnels

5.2.1 Test 1 : Création d'une idée

Nous avons vérifié qu'il est possible de créer une idée. Le statut par défaut est bien "Brouillon".

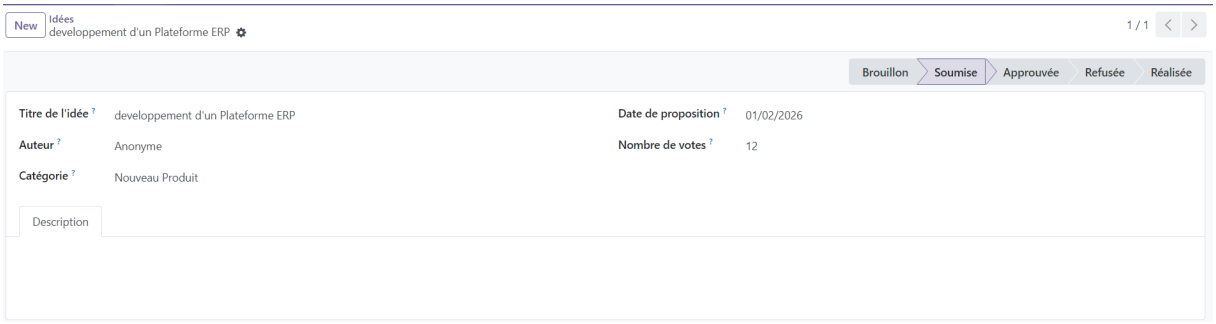


FIGURE 5.2 – Saisie d'une nouvelle idée

5.2.2 Test 2 : Validation (Workflow)

En cliquant sur la barre d'état, nous avons passé l'idée en statut "Validée". La vue liste reflète ce changement avec une couleur verte (attribut `decoration-success`).

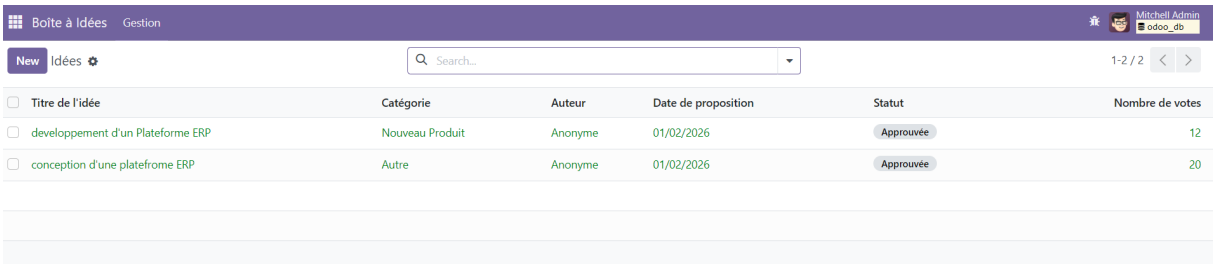


FIGURE 5.3 – Liste des idées avec différents statuts

Chapitre 6

Conclusion Générale et Bilan

Au terme de ce projet académique consacré à la conception, au développement et au déploiement d'un module métier sous l'ERP Odoo 17, il convient de dresser un bilan exhaustif des travaux réalisés, tant sur le plan technique que méthodologique. La problématique initiale consistait à digitaliser le processus de gestion de l'innovation participative, souvent traité de manière informelle dans les entreprises, via une solution structurée, pérenne et sécurisée.

6.1 Bilan Technique et Architectural

L'un des piliers de ce projet a été le choix d'une architecture décorrélée du système d'exploitation hôte. L'utilisation de la conteneurisation via Docker et l'orchestration via Docker Compose ne s'est pas limitée à une simple facilité d'installation. Elle a permis de simuler un environnement de production réel où la base de données (PostgreSQL 16) et le serveur d'application (Odoo 17) fonctionnent comme des micro-services distincts, communiquant via un réseau virtuel privé. Cette approche a mis en évidence l'importance de la gestion des volumes persistants pour garantir l'intégrité des données malgré le caractère éphémère des conteneurs.

Concernant le développement spécifique, l'immersion dans le framework Odoo a révélé la puissance de son ORM (Object-Relational Mapping). L'abstraction de la couche SQL via les modèles Python a permis de se concentrer sur la logique métier plutôt que sur les requêtes de bas niveau. La définition de la classe `boite.idee` et de ses attributs typés (Selection, Date, Char) a démontré comment Odoo gère automatiquement la création des tables et des contraintes d'intégrité en base de données.

De plus, la construction de l'interface utilisateur via les vues XML (Tree et Form) a souligné l'efficacité de l'architecture MVC. La séparation stricte entre les données (Modèle), l'interface (Vue) et la logique de contrôle a facilité le débogage et garantit une

maintenabilité accrue du code.

6.2 Bilan Fonctionnel et Ergonomique

La solution délivrée, bien que constituant un MVP (Minimum Viable Product), couvre l'intégralité du périmètre fonctionnel défini lors de la phase d'analyse. Le module "Boîte à Idées" permet :

- Une saisie normalisée des propositions, éliminant les ambiguïtés liées aux formats hétérogènes (emails, papier).
- Une traçabilité complète grâce à l'historisation automatique des créations et modifications (champs magiques Odoo).
- Une gestion rigoureuse du cycle de vie de l'idée, passant d'un état instable (Brouillon) à un état arbitré (Validée/Refusée) via un workflow contrôlé.

L'intégration native dans l'interface web d'Odoo assure une prise en main immédiate pour les utilisateurs, sans rupture ergonomique avec les autres applications de l'ERP.

6.3 Apports Personnels et Compétences Acquises

Ce projet a été un vecteur d'apprentissage majeur. Il a permis de :

1. **Maîtriser la chaîne de déploiement Docker :** Compréhension des *bind mounts*, des variables d'environnement et des logs conteneurisés.
2. **Acquérir une rigueur de développement :** Respect des conventions de nommage Python (PEP8) et structure standardisée des modules Odoo.
3. **Comprendre la sécurité applicative :** La configuration des ACL (Access Control Lists) a sensibilisé à l'importance de la protection des données et de la gestion des privilèges.
4. **Gérer les imprévus :** La résolution des erreurs de configuration (notamment l'erreur 500 liée aux chemins de fichiers) a renforcé les capacités de diagnostic et de dépannage (troubleshooting).

En conclusion, ce projet valide la pertinence d'Odoo comme framework de développement rapide (RAD) pour les applications de gestion, capable d'allier robustesse technique et flexibilité fonctionnelle.

Chapitre 7

Perspectives d'Évolution et Industrialisation

Le module actuel pose les fondations solides d'un système de gestion de l'innovation. Cependant, pour passer d'un projet académique à une solution déployable à grande échelle dans une entreprise, plusieurs axes d'amélioration et d'industrialisation doivent être envisagés.

7.1 Évolutions Fonctionnelles Avancées

7.1.1 Intégration Transverse (Cross-Module)

La force d'un ERP réside dans l'interconnexion de ses modules. Une évolution majeure consisterait à lier la "Boîte à Idées" au module "Projets" d'Odoo.

— **Automatisation :** Lorsqu'une idée passe au statut "Validée", un "Projet" ou une "Tâche" pourrait être automatiquement créé(e) dans le module de gestion de projets, avec l'auteur de l'idée assigné comme observateur.

— **Lien RH :** L'intégration avec le module "Employés" permettrait de lier automatiquement l'idée au profil RH de l'utilisateur connecté (via `res.users`), évitant la saisie manuelle du nom de l'auteur.

7.1.2 Communication et Collaboration (Chatter)

L'ajout du mixin `mail.thread` au modèle `boite.idee` est une priorité. Cela activerait la zone de communication (le "Chatter") en bas de chaque fiche.

— **Historique :** Trace de tous les changements d'états avec horodatage.

— **Collaboration :** Possibilité pour les managers et employés d'échanger des commentaires directement sur la fiche de l'idée.

— **Notifications :** Envoi automatique d'emails lors des validations ou refus.

7.1.3 Gamification et Engagement

Pour encourager l'adoption, le module pourrait utiliser le moteur de gamification d'Odoo.

— Création de "Badges" (ex : "Innovateur du mois").
— Attribution de points pour chaque idée validée.
— Mise en place d'un classement (Leaderboard) des contributeurs les plus actifs.

7.2 Améliorations Techniques et Sécurité

7.2.1 Sécurité Granulaire (Record Rules)

Actuellement, la sécurité est gérée au niveau du modèle (ACL). Pour une version industrielle, il faudrait implémenter des "Règles d'enregistrement" (Record Rules).

— **Règle utilisateur :** Un employé ne doit voir et modifier que ses propres idées (filtre `[('create_uid', '=', user.id)]`).
— **Règle manager :** Un manager doit pouvoir voir toutes les idées mais ne modifier que celles de son département.

7.2.2 Portail Web (Frontend)

Pour toucher les collaborateurs qui n'ont pas accès au back-office Odoo (ouvriers, terrain), le développement d'une interface web via le module "Website" est pertinent. Cela impliquerait la création de contrôleurs Python (`controllers/main.py`) et de templates QWeb pour offrir un formulaire de soumission simplifié et responsive, accessible depuis un smartphone.

7.3 Stratégie de Déploiement (DevOps)

7.3.1 Intégration Continue (CI/CD)

L'industrialisation du module passerait par la mise en place d'un pipeline CI/CD (GitHub Actions ou GitLab CI).

— **Linting :** Vérification automatique de la qualité du code (Pylint, Flake8).

— **Tests Unitaires :** Écriture de tests automatisés via la librairie `odoo.tests` pour garantir que le workflow de validation fonctionne après chaque modification de code.

7.3.2 Hébergement Cloud

Enfin, la migration de l'architecture Docker locale vers une solution Cloud (AWS, Azure ou Odoo.sh) assurerait la haute disponibilité du service, avec des mécanismes de sauvegarde automatique de la base PostgreSQL et de monitoring des performances.

En somme, ce projet ouvre la voie à de nombreuses possibilités d'extension, illustrant la richesse de l'écosystème Odoo et sa capacité à s'adapter aux besoins métiers les plus spécifiques.

Bibliographie

— Odoo S.A. (2024). *Odoo 17 Developer Documentation*.
— AIT DAOUD, M. (2024). *LAB 2 : Création d'un module Odoo*. Support de cours EMSI.
— AIT DAOUD, M. (2024). *L'ERP : Formation Pédagogique Pratique avec Odoo*. Support de cours EMSI.
— Docker Inc. (2024). *Docker Compose Documentation*.